ProManage Bot: Workflow

Table of Contents

1. Modules of the Project 2
    1. Integration with Jira / Other Tools 2
    2. Risk Analysis & Prediction 2
    3. Agentic AI (Decision Executor) 2
    4. Chatbot Interface 3
    5. Suggestions Board 3
    6. Rule-Based Validation Engine 3
    7. Explainability & Reasoning Module 4
    8. Monitoring & Alerting Module 4
    9. Audit & Decision History Module 4
    10. Data Preprocessing & Normalization Module 5
2. 8-Week Agile Development Plan 5
    Week 1: Project Initialization and Core Setup 5
    Week 2 – Project Data Integration and Ingestion 6
    Week 3 – Data Preprocessing and Rule-Based Validation 7
    Week 4 – Risk Analysis and Prediction Module 7
    Week 5 – Agentic AI Decision Engine 8
    Week 6 – AI Recommendations and Suggestions Board 9
    Week 7 – Chatbot Interface and Monitoring System 9
    Week 8 – Testing, Optimization, and Final Deployment 10

1. Modules of the Project

1. Integration with Jira / Other Tools

Purpose:

- Fetch real-time project data (tasks, sprints, assignees, deadlines)
- Keep the system synced with actual project state

Why it's important:

- Makes the project realistic and industry-aligned
- Removes manual data entry

2. Risk Analysis & Prediction

Purpose:

- Detect risks like:
  - Delays
  - Overloaded team members
  - Bottleneck tasks
- Use historical + live data

Tech angle:

- ML models + rule-based checks
- Core "intelligence" of the system

3. Agentic AI (Decision Executor)

Purpose:

- Acts as the brain of the system
- Monitors data continuously
- Coordinates between:
  - Rules engine
  - AI models

    - External APIs

4. Chatbot Interface

Purpose:

- Natural language interaction
- PM can ask:
    - "Why is this task delayed?"
    - "What should I fix first?"

Value:

- Improves usability
- Makes AI explainable (very important for viva)

5. Suggestions Board

Purpose:

- Central place to view:
    - AI recommendations
    - Reasons behind them
    - Approve / Reject buttons

6. Rule-Based Validation Engine

Even though it's mentioned inside other modules, making it a separate module is smart.

What it does:

- Enforces Scrum, PMBOK, workload limits
- Blocks invalid AI suggestions

Why it matters:

- Shows hybrid AI (Rules + ML)
- Protects against hallucinated decisions

7. Explainability & Reasoning Module

Purpose:

- Converts raw AI output into:
    - Clear reasons
    - Bullet-point justifications
- Example:

"Task reassigned because user A is 140% overloaded"

8. Monitoring & Alerting Module

Purpose:

- Continuous background monitoring
- Sends alerts for:
    - High-risk tasks
    - Deadline breaches
    - Sprint failure probability

Why it's good:

- Proactive, not reactive
- Makes the system feel "alive"

9. Audit & Decision History Module

Purpose:

- Logs:
    - AI suggestions
    - PM approvals/rejections
    - Actions executed

Huge plus for:

- Accountability

- Debugging
- Academic evaluation

10. Data Preprocessing & Normalization Module

Purpose:

- Cleans data from Jira/Trello
- Converts everything into one standard format

Why separate module helps:

- Clean architecture
- Easier testing & grading

---

2. 8-Week Agile Development Plan

Total Duration: 8 Weeks
Methodology: Agile (Iterative and Incremental Development)
Approach: Core system first, intelligence and automation added step by step
Goal: A functional system at the end of every 1–2 weeks

Week 1: Project Initialization and Core Setup

Objectives

- Establish a strong technical foundation
- Define system architecture and development workflow
- Ensure backend and database are operational

Features to Implement

- Project folder structure and modular architecture
- Backend server setup
- Database connection
- Basic user authentication for project manager

Deliverables

- Backend server running successfully

- MongoDB connected and accessible
- Login API functional
- Git repository initialized with proper commits
- Basic API testing completed

Tools and Technologies

- Backend: Python (FastAPI) or Node.js (Express)
- Database: MongoDB
- Authentication: JWT
- Version Control: Git, GitHub
- API Testing: Postman

Week 2 – Project Data Integration and Ingestion

Objectives

- Enable the system to work with real project data
- Support both manual and automated data sources

Features to Implement

- Project data upload via JSON
- Jira API integration for fetching live project data
- Data validation and normalization
- Store structured task data in database

Deliverables

- Successful project data upload
- Jira tasks fetched and stored
- Clean and standardized data format
- API endpoints for viewing project data

Tools and Technologies

- Jira REST API
- Backend framework (FastAPI or Express)

- Data validation libraries (Pydantic or Joi)
- MongoDB schemas

Week 3 – Data Preprocessing and Rule-Based Validation

Objectives

- Ensure correctness and reliability of data
- Enforce project management constraints

Features to Implement

- Rule-based validation engine using configurable rules
- Validation of workload limits, deadlines, and dependencies
- Error handling and violation reporting
- Logging of validation results

Deliverables

- Rule engine fully operational
- Detection of rule violations
- Invalid workflows blocked
- Configurable rules stored in JSON

Tools and Technologies

- Rule configuration using JSON
- Backend validation logic
- Logging framework

Week 4 – Risk Analysis and Prediction Module

Objectives

- Identify project risks early
- Provide analytical insights to the system

Features to Implement

- Risk indicators for delays and overload
- Historical and current data analysis
- Simple machine learning or heuristic models
- Risk scoring for tasks and sprints

Deliverables

- Risk prediction results stored in database
- Risk flags visible through APIs
- Delay and overload predictions generated

Tools and Technologies

- Python (Pandas, NumPy)
- Machine Learning: scikit-learn
- MongoDB for storing risk metrics

Week 5 – Agentic AI Decision Engine

Objectives

- Build the autonomous decision-making layer
- Coordinate analysis, rules, and recommendations

Features to Implement

- Agentic workflow (observe, analyze, decide, suggest)
- Decision logic based on risks and rules
- Internal action planning
- Decision trace storage for explainability

Deliverables

- Agentic AI pipeline functioning
- Decision logic producing suggestions
- Reasoning data stored for transparency

Tools and Technologies

- Custom agent logic in Python

- Background task scheduling
- Database-driven state management

Week 6 – AI Recommendations and Suggestions Board

Objectives

- Present intelligent recommendations to the user
- Introduce human-in-the-loop control

Features to Implement

- AI-generated recommendations using Gemini API
- Explainable reasoning for each suggestion
- Suggestions board interface
- Approve and reject functionality

Deliverables

- AI recommendations visible on dashboard
- Clear explanations for decisions
- Human approval workflow operational
- Approved actions marked for execution

Tools and Technologies

- Gemini API
- Frontend: React
- State management: Context API or Redux

Week 7 – Chatbot Interface and Monitoring System

Objectives

- Enable natural language interaction
- Implement continuous project monitoring

Features to Implement

- Chatbot interface for project queries
- Natural language question answering

- Risk alerts and notifications
- Continuous monitoring service

Deliverables

- Chatbot responding to project-related queries
- Alerts triggered for high-risk situations
- Monitoring processes running in background
- Alert logs stored in database

Tools and Technologies

- Gemini API for conversational responses
- React-based chat interface
- Backend event handling or polling mechanisms

Week 8 – Testing, Optimization, and Final Deployment

Objectives

- Stabilize the system
- Prepare the project for evaluation and demonstration

Features to Implement

- End-to-end system testing
- UI and UX improvements
- Performance optimization
- Audit logs and decision history
- Final deployment

Deliverables

- Fully functional integrated system
- All test cases passed
- Deployed application
- Final documentation and demo-ready product

Tools and Technologies

- Testing: PyTest or Jest

- Deployment: Docker, Render, or Railway
- Documentation: README, SRS updates, system diagrams